INFORME TÉCNICO: EVALUACIÓN INTEGRADORA PARTE 1

Detección de Objetos, Identificación de Vehículos Rojos, Sincronización Multi-Cámara y Despliegue Reproducible Zero-Touch en Producción

Materia / Institución:	Visión por Computadora — Lic. en Inteligencia Artificial Universidad Blas Pascal (UBP)
Integrantes:	Dennis Nelson Fernández (Legajo 189692) Marcos Andrés Wamba Ferreyra (Legajo 190802)
Docente a cargo:	César Osimani
Repositorio Oficial & Colab:	GitHub: https://github.com/fernandezdennis/vc-integrador-ubp Ejecución Zero-Touch en Google Colab

1. Introducción y Objetivos del Proyecto

El despliegue de sistemas inteligentes de monitoreo de tránsito urbano en intersecciones complejas con rotondas enfrenta desafíos operacionales de alta exigencia: heterogeneidad de cámaras de videovigilancia, variaciones de latencia en la transmisión por red y oclusiones geométricas dinámicas generadas por la infraestructura y el flujo vehicular. El uso de múltiples cámaras sincronizadas permite extender la cobertura visual mediante perspectivas complementarias: vista frontal y rasante a nivel de calle (calle), vista elevada desde edificio (piso3) y vista panorámica lateral municipal (muni). Sin embargo, cualquier etapa subsiguiente de re-identificación o seguimiento global de trayectorias (Multi-Target Multi-Camera Tracking - MTMC) requiere estrictamente una alineación temporal precisa y una extracción de características visuales estructurada y robusta.

Los objetivos centrales de esta primera etapa comprenden:

- **Sincronización multi-cámara:** Adquirir, normalizar y sincronizar temporalmente tres transmisiones de video urbanas mediante análisis cinemático y edición determinista con FFmpeg.
- **Optimización operativa de producción:** Determinar técnica y cinemáticamente la frecuencia de muestreo (15 FPS) y la resolución de trabajo (1280×720 en captura, 640×640 en inferencia) requeridas para escalar a 15 cámaras en producción.
- **Arquitectura de red municipal y MediaMTX:** Auditar el protocolo de streaming HLS y fundamentar el rol de MediaMTX como broker multimedia local de latencia ultrabaja (<500 ms) para ingestión de IA.
- **Dimensionamiento de hardware:** Diseñar la arquitectura de hardware municipal para procesar concurrentemente 5 intersecciones (15 cámaras a 15 FPS = 225 FPS globales).
- **Percepción visual con YOLOv8:** Detectar y clasificar actores viales relevantes ('person', 'car', 'motorcycle', 'bicycle', 'bus', 'truck') en las tres perspectivas concurrentes.
- **Filtrado de falsos positivos por ROIs:** Implementar zonas espaciales de exclusión mediante camara_rois.py para suprimir detecciones erróneas en objetos estáticos fuera de la calzada.
- **Discriminación cromática de vehículos rojos:** Desarrollar un clasificador en espacio HSV combinado con dispersión espacial (spread) para aislar carrocerías rojas y descartar luces de freno y artefactos.
- **Trazabilidad y unificación de datos:** Generar registros tabulares individuales y consolidarlos en un único DataFrame unificado (df_total / eventos_total.csv).
- **Despliegue MLOps Zero-Touch:** Garantizar la reproducibilidad automatizada de extremo a extremo en Google Colab con código versionado en GitHub y recursos alojados en Google Drive público descargables vía gdown.

2. Arquitectura del Repositorio y Reproducibilidad Zero-Touch

Siguiendo las mejores prácticas de MLOps y las pautas de evaluación académica, se eliminó toda dependencia de montajes interactivos manuales de Google Drive personal. El sistema fue desacoplado en dos capas operativas que permiten la ejecución 'Zero-Touch' (un solo clic en 'Ejecutar todas' en Google Colab sin intervención del usuario):

Capa de Arquitectura	Plataforma / Repositorio	Función y Contenido
Código Fuente y Configuración	GitHub (Repositorio Público)	Contiene módulos Python (/scripts/), data.yaml (/configs/), requirements.txt y el cuaderno Jupyter orquestador. Permite versionado liviano y despliegue continuo.
Almacenamiento Pesado de Recursos	Google Drive Público (Vía gdown)	Aloja recursos binarios en recursos_vc.zip (pesos preentrenados best.pt, yolov8n.pt, dataset_v2 y videos de producción en /videos_prod/). Descarga automatizada sin autenticación.

Al ejecutar la Celda 1 del cuaderno en Google Colab, se activa el pipeline de aprovisionamiento automatizado:

```
# 1. Clonado del repositorio público de GitHub
git clone https://github.com/fernandezdennis/vc-integrador-ubp /content/proyecto_vc
pip install -q -r /content/proyecto_vc/requirements.txt

# 2. Descarga del paquete público mediante gdown (ID público del ZIP)
gdown.download(id='1LYbi_6j_vkI3yHYAu3ZWEFRjRxhW6PRt', output='/content/data/recursos_vc.zip')
!unzip -q /content/data/recursos_vc.zip -d /content/data

# 3. Configuración dinámica de rutas en data.yaml y variables de sesión
config['path'] = '/content/data/dataset_v2'
```

3. Captura, Protocolos de Video y Rol de MediaMTX en Producción

Las tres perspectivas de monitoreo corresponden a cámaras urbanas de la red pública de videovigilancia de Neuquén Capital, situadas en torno al monumento a San Martín. El análisis de los canales de acceso originales revela particularidades críticas que deben atenderse para la producción municipal:

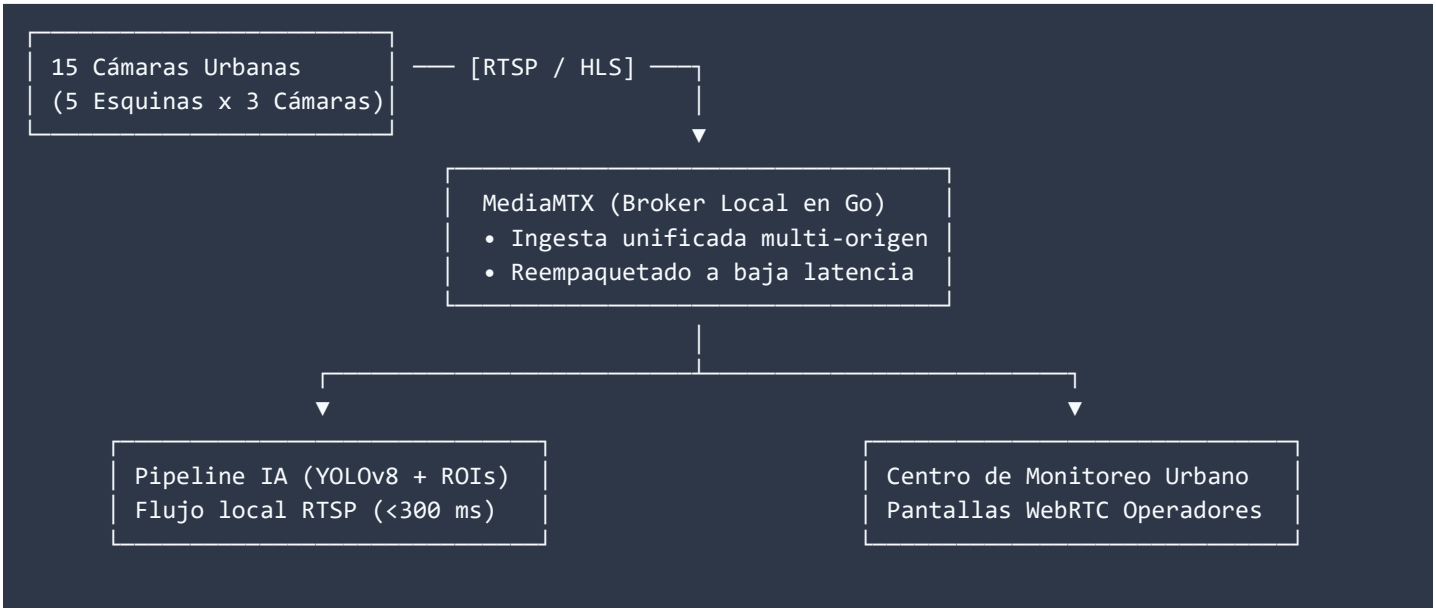
3.1 Diagnóstico del Protocolo HLS Actual

1. Mecanismo de transmisión: Las cámaras transmiten mediante el protocolo HTTP Live Streaming (HLS) a través de listas de reproducción .m3u8 que segmentan el flujo de video en archivos .ts.
2. Latencia acumulada severa: HLS está diseñado para distribución tolerante a fallas en redes de consumo masivo, manteniendo buffers de 2 a 6 segundos por segmento. En la práctica, esto genera una latencia de 6 a 30 segundos respecto a los sucesos en calzada.
3. Deriva temporal asíncrona: Cada cliente HTTP administra su propia memoria intermedia de forma independiente, ocasionando que las tres cámaras presenten desfases dinámicos de varios segundos entre sí.
4. Inexistencia de reloj de hardware unificado: La ausencia de sincronismo por hardware (PTP/NTP) en los streams públicos demanda una etapa de normalización y alineación cinemática previa a la inferencia.

3.2 Rol y Beneficios de MediaMTX como Broker de Producción

MediaMTX (anteriormente rtsp-simple-server) es un broker multimedia de código abierto, ultraligero y de alto rendimiento programado en Go. En una arquitectura municipal de escala real, MediaMTX actúa como la capa de transporte intermedia entre las cámaras físicas y los modelos de visión por computadora:

- **Redistribución local a latencia ultrabaja (<500 ms):** Ingesta flujos RTSP/HLS directos de las cámaras y los convierte en streams locales RTSP sobre TCP o WebRTC de latencia despreciable, habilitando procesamiento de IA en tiempo real.
- **Optimización drástica de ancho de banda:** Evita la saturación del enlace de subida de la vía pública. La cámara emite un único flujo hacia el centro de datos local de MediaMTX, y este replica el flujo (1 a N) hacia los scripts de YOLOv8 y los operadores de monitoreo.
- **Estandarización de endpoints lógicos:** Homologa las fuentes en URLs coherentes y mantenibles (ej. rtsp://mediamtx:8554/esquina1/calle, .../piso3, .../muni).



4. Configuración Operativa, Definición de FPS y Hardware Municipal

Durante las etapas preliminares de laboratorio, las secuencias de video fueron codificadas a 30 FPS para facilitar la inspección visual cuadro por cuadro. Sin embargo, a partir de las devoluciones de la cátedra orientadas a una implantación real en el municipio, se reevaluaron rigurosamente los parámetros operativos.

4.1 Justificación Cinemática y Computacional de los 15 FPS Finales

Se definió establecer la tasa temporal definitiva de operación en 15 FPS basándose en tres fundamentos técnicos clave:

1. Cinemática de tránsito en intersecciones: En rotondas y esquinas urbanas, la velocidad de circulación oscila entre 30 y 40 km/h (8.33 a 11.11 m/s). A 15 FPS (intervalo $\Delta t = 66.6$ ms entre fotogramas sucesivos), un automóvil recorre entre 0.55 y 0.74 metros por cuadro. Esta distancia es considerablemente menor a la longitud de los vehículos (3.5 - 5.0 metros), garantizando una superposición espacial (IoU > 0.65) plenamente continua entre cuadros para tracking y asociación de trayectorias sin pérdida de identidad.
2. Reducción a la mitad del costo computacional: Procesar 15 cámaras concurrentes a 30 FPS implicaría una tasa de 450 FPS ininterrumpidos. A 15 FPS, la exigencia se reduce exactamente a 225 FPS globales (un ahorro computacional del 50%), permitiendo que una única estación con GPU avanzada aloje todo el sistema sin recurrir a costosos centros de datos externos.
3. Ausencia de movimientos de alta velocidad: Dado que el proyecto no involucra reconocimiento facial ni dinámicas

deportivas de alta aceleración, 15 FPS representa el punto de equilibrio óptimo entre precisión perceptiva y viabilidad de hardware.

4.2 Estandarización de Resoluciones

• Resolución de Streaming y Captura (1280×720 píxeles - HD): Preserva la fidelidad de siluetas lejanas y características cromáticas en tomas panorámicas (piso3) sin sobrecargar el ancho de banda con flujos 1080p o 4K innecesarios.
• Resolución de Inferencia del Modelo (imgsz = 640×640 píxeles): Dimensión estándar de YOLOv8 optimizada con padding adaptativo (letterbox). Proporciona la mayor métrica mAP a latencias inferiores a 10 ms por imagen en GPU.

4.3 Dimensionamiento de Hardware Municipal (15 Cámaras / 5 Intersecciones)

Para procesar en simultáneo 15 cámaras urbanas (5 esquinas) a 15 FPS (225 FPS de inferencia total + decodificación de video + filtrado de color y ROIs), se define la siguiente estación de trabajo homologada con la Sección 4 del Colab:

Componente	Especificación Propuesta	Justificación Técnica (15 Cámaras Concurrentes)
GPU (Procesamiento Gráfico)	NVIDIA GeForce RTX 4080 Super (16 GB GDDR6X) o RTX A4500	Procesa holgadamente 225 FPS globales. YOLOv8n en FP16 requiere ~3 ms/frame en batch; 16 GB de VRAM alojan tensores y decodificación NVDEC para 15 canales.
CPU (Procesador Central)	AMD Ryzen 9 9950X (16 núcleos / 32 hilos) o Xeon equivalente	Gestiona decodificación multihilo de video y preprocesamiento de OpenCV (cálculo de máscaras HSV, spread espacial y polígonos ROI) en paralelo.
Memoria RAM	64 GB DDR5 5600 MHz	Garantiza buffers de cuadros continuos para 15 streams de video simultáneos sin riesgo de desbordamiento (OOM).
Almacenamiento SO / IA	SSD NVMe 2 TB PCIe 4.0 (lectura >5000 MB/s)	Carga instantánea de pesos, entornos y almacenamiento de bases de datos intermedias y logs de detección.
Almacenamiento de Video	RAID 1 con 2 discos HDD de 4 TB a 8 TB para videovigilancia	Grabación circular de evidencias en video de las 15 cámaras y volcados históricos de CSV/SQL durante 30 a 60 días.
Sistema Operativo	Ubuntu Server 24.04 LTS (x86_64) con NVIDIA Container Toolkit	Plataforma industrial para despliegue modular con Docker, drivers CUDA estables y ejecución continua sin sobrecarga de escritorio gráfico.

5. Preprocesamiento, Sincronización y Configuración de Pruebas

La sincronización de las secuencias se ejecutó mediante un protocolo matemático y visual riguroso en dos etapas:
1. Normalización de tasa de fotogramas a 15 FPS constantes mediante FFmpeg (libx264, CRF 18).
2. Determinación de offsets temporales de inicio tomando la cámara municipal (muni) como eje temporal absoluto de referencia.

5.1 Desfases Temporales y Comandos de Corte

A partir de eventos cinemáticos deterministas (paso de vehículos sobre demarcaciones peatonales de la rotonda), se registraron las marcas temporales de alineación:
• Desfase Calle: Δt = 31.100 s (recorte de 933 cuadros a 30 FPS / 466 cuadros a 15 FPS) -> calle_prod.mp4.

• Desfase Piso 3: Δt = 7.233 s (recorte de 217 cuadros a 30 FPS / 108 cuadros a 15 FPS) -> piso3_prod.mp4.
• Muni: Secuencia base de referencia sin recorte inicial -> muni_prod.mp4.

```
# Comandos de normalización a 15 FPS y recorte determinista (PTS):
ffmpeg -i calle_raw.mp4 -vf 'trim=start=31.10,setpts=PTS-STARTPTS,fps=15' -c:v libx264 -crf 18
calle_prod.mp4
ffmpeg -i piso3_raw.mp4 -vf 'trim=start=7.233,setpts=PTS-STARTPTS,fps=15' -c:v libx264 -crf 18
piso3_prod.mp4
ffmpeg -i muni_raw.mp4 -vf 'fps=15' -c:v libx264 -crf 18 muni_prod.mp4
```

5.2 Modalidad de Ejecución: Videos de Producción vs. Videos Cortos

Tal como se programó en la Celda 4 del Colab, el pipeline implementa un selector paramétrico USE_SHORT_VIDEOS que permite alternar entre los videos completos de 10 minutos (tiempo de corrida de ~25 minutos) y secuencias recortadas de 1 minuto para validaciones rápidas de código:

Cámara ID	Perspectiva Urbana	Archivo Producción (Largo)	Archivo Prueba (1 Minuto)
calle	Vista frontal rasante a nivel de calle	calle_prod.mp4	calle_prod1min.mp4
piso3	Vista cenital elevada desde edificio	piso3_prod.mp4	piso3_prod1min.mp4
muni	Vista panorámica lateral municipal (Base)	muni_prod.mp4	muni_prod1min.mp4

6. Inferencia con YOLOv8, Filtrado Espacial de ROIs y Detección Cromática

El software se diseñó bajo una arquitectura modular y desacoplada en la carpeta /scripts/:
• pipeline_generico.py: Motor común de inferencia que carga el modelo YOLOv8n, consulta zonas de exclusión y analiza color.
• pipeline_calle.py, pipeline_piso3.py, pipeline_muni.py: Wrappers especializados para cada cámara.
• camara_rois.py: Módulo geométrico con definición de polígonos de exclusión espacial por perspectiva.
• utils_color.py: Clasificador cromático en espacio HSV con análisis de dispersión espacial (spread).
• Clases viales evaluadas (CLASSES_INTERES): person, car, motorcycle, bicycle, bus, truck.

6.1 Filtrado Espacial de Falsos Positivos por Regiones de Interés (ROIs)

En escenas urbanas reales, existen estructuras estáticas fuera de la calzada que provocan falsos positivos reiterados en detectores neuronales (ej. contenedores metálicos de obra clasificados como camiones/autos, o estatuas ecuestres clasificadas como vehículos). Mediante la función inside_exclusion_zone(camera, xc, yc) en camara_rois.py, el pipeline descarta activamente detecciones cuyos centros caigan en polígonos no transitables. En la salida de ejecución del Colab (Celda 9), se registraron y descartaron cientos de falsos positivos en la cámara municipal:

```
# Registro de exclusiones activas en la salida de consola de Colab:
ROI HIT muni contenedor_obra 0.78288 0.48605 -> EXCLUIDO -> muni car (0.783, 0.486)
ROI HIT muni monumento_san_martin 0.63290 0.46134 -> EXCLUIDO -> muni car (0.633,
0.461)
```

6.2 Clasificación Robusta de Vehículos Rojos (HSV + Spread)

El análisis clásico de color por umbral de píxeles genera severos falsos positivos debido a las luces traseras de freno encendidas en automóviles blancos o negros. El algoritmo desarrollado en utils_color.py combina dos condiciones necesarias y complementarias:

1. Máscara cromática HSV dual: Evalúa los dos extremos del matiz rojo en HSV ($H \in [0, 10]$ y $[170, 180]$) con umbrales de saturación $S > 70$ y valor $V > 50$.
2. Dispersión espacial (Spread): Mide la desviación estándar de la distribución espacial de los píxeles rojos dentro de la caja delimitadora. Las luces de freno generan focos muy concentrados (bajo spread, spread < 0.25), mientras que una carrocería o capó rojo presenta una distribución amplia y dispersa (spread ≥ 0.25). Esto garantiza la correcta identificación de vehículos genuinamente rojos.

7. Integración Multi-Cámara y Dataset Consolidado (df_total)

Cada pipeline individual procesa su video y genera un archivo CSV con las detecciones registradas por fotograma. Posteriormente, la Celda 10 del Colab ejecuta la concatenación de los tres dataframes, generando df_total y exportándolo a eventos_total.csv:

```
df_total = pd.concat([df_calle, df_piso3, df_muni], ignore_index=True)
df_total.to_csv(os.path.join(RESULTADOS_DIR, 'eventos_total.csv'), index=False)
```

Estructura de atributos exportada en los archivos CSV:

Columna	Tipo de Dato	Descripción Técnica
frame_id	Integer	Índice secuencial del fotograma procesado en la secuencia de video.
timestamp_s	Float	Tiempo relativo en segundos (calculado como frame_id / 15.0).
camera	String	Identificador de la perspectiva de origen: 'calle', 'piso3' o 'muni'.
class	String	Clase vehicular o peatonal clasificada por YOLOv8 ('car', 'person', etc.).
confidence	Float	Puntaje de confianza de la detección [0.0 - 1.0].

Distribución estadística obtenida en la ejecución completa de las tres cámaras (total: 16.381 detecciones consolidadas):

Cámara / Fuente	Cantidad de Detecciones	Archivo Tabular Generado
muni (Vista Municipal)	7.198 eventos (43.9%)	eventos_muni.csv
piso3 (Vista Elevada)	6.496 eventos (39.7%)	eventos_piso3.csv
calle (Vista Frontal)	2.687 eventos (16.4%)	eventos_calle.csv
TOTAL CONSOLIDADO (df_total)	**16.381 detecciones unificadas**	**eventos_total.csv**

Visualización analítica generada en el entorno interactivo:

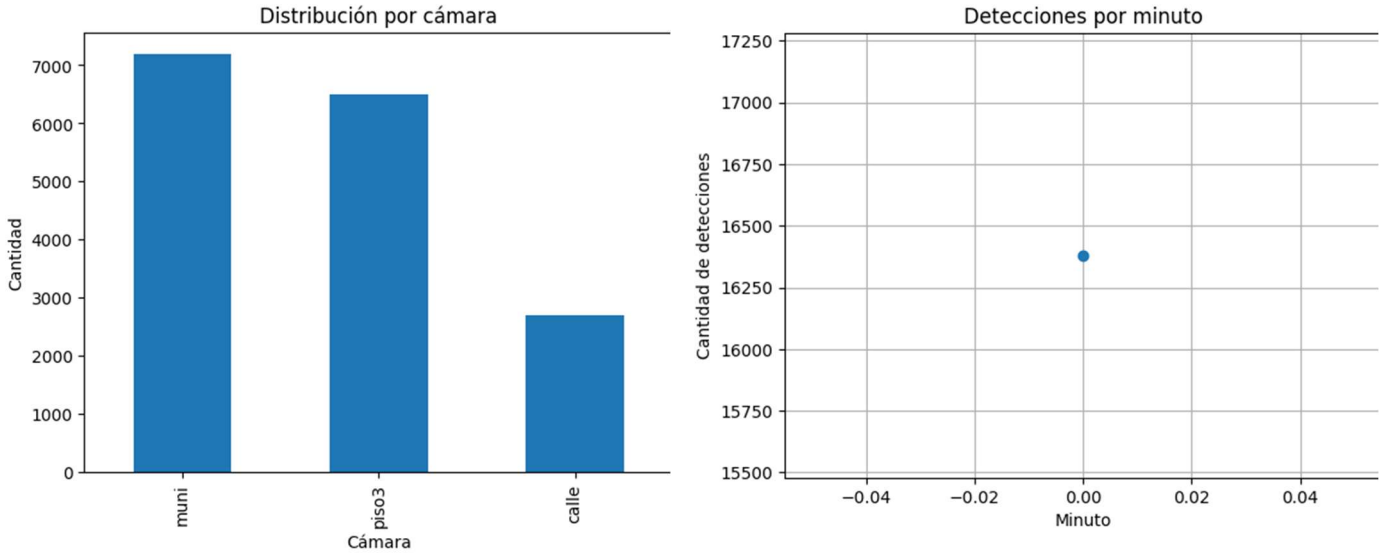


Figura 1: Distribución total de detecciones por cámara (izquierda) y distribución por clase (derecha).

8. Evidencia Visual y Diagnóstico de Inferencia

Como respaldo cualitativo auditable de la sincronización y la detección del sistema, se configuró la extracción gráfica simultánea de fotogramas. El Colab implementa la selección de evidencia en función del tipo de ejecución:

• FRAME_TO_SHOW = 196: Frame representativo para ejecuciones rápidas en videos cortos (USE_SHORT_VIDEOS = True).

• FRAME_TO_SHOW = 1892 (o 930): Frame de auditoría para secuencias completas sincronizadas.

Resultados de inferencia obtenidos en la consola de Colab en el fotograma 196 (Celda 12):

• calle: 1 person, 3 cars (Velocidad: 226.1 ms inferencia, 14.0 ms preproceso).

• piso3: 1 person, 10 cars, 1 bus (Velocidad: 130.6 ms inferencia, 5.2 ms preproceso).

• muni: 2 persons, 6 cars, 1 bus (Velocidad: 135.9 ms inferencia, 4.3 ms preproceso).

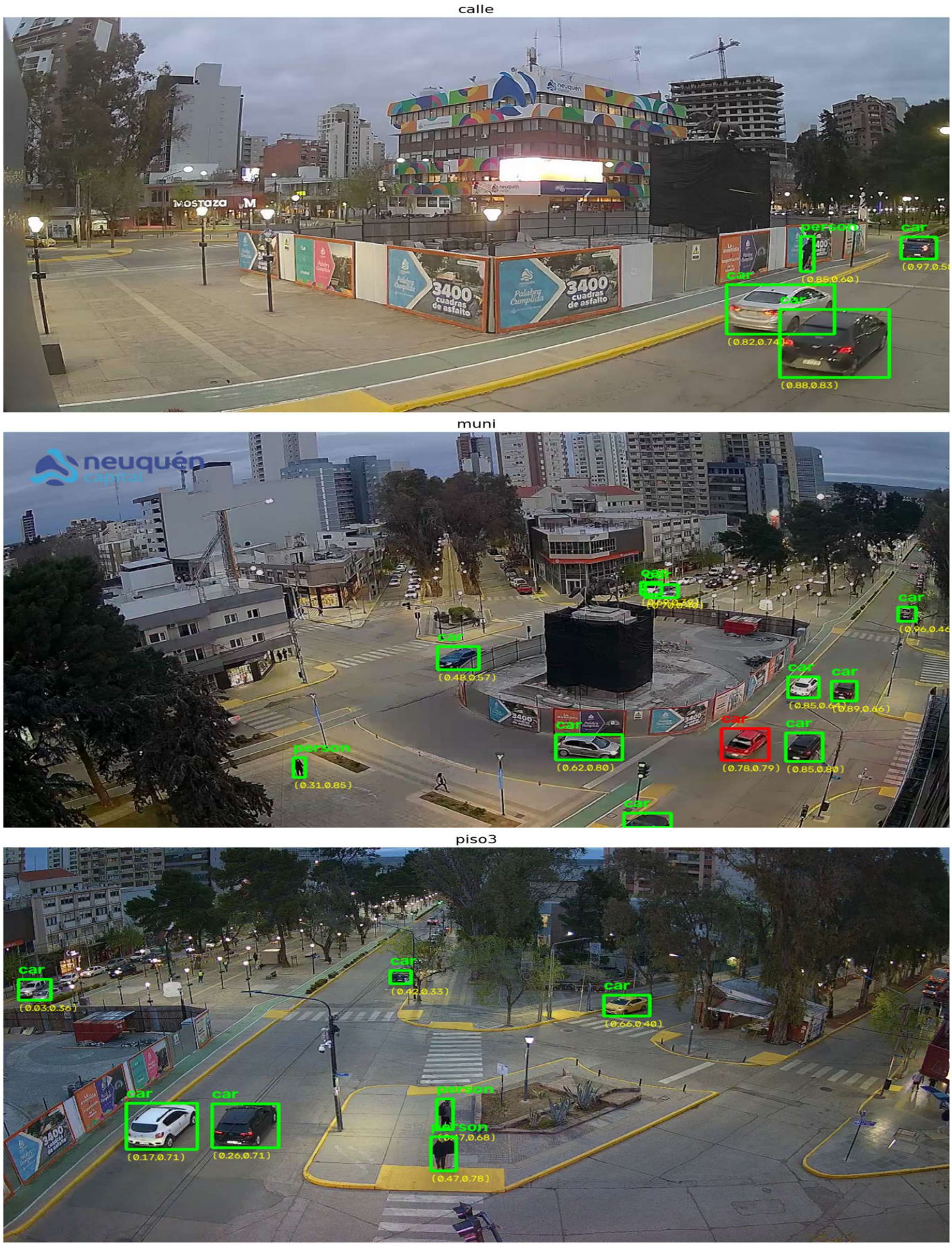


Figura 2: Captura sincronizada simultánea en las tres cámaras (calle, muni, piso3) con Bounding Boxes, coordenadas y clasificación de vehículo rojo (is_red=True en recuadro rojo).

9. Metodología de Trabajo y División de Responsabilidades MLOps

El proyecto se desarrolló bajo una dinámica colaborativa e interdisciplinaria que articuló la ingeniería algorítmica y la auditoría rigurosa de calidad:

Integrante / Rol	Responsabilidades y Aportes Técnicos Principales
Dennis Nelson Fernández (Legajo 189692) Arquitectura MLOps e Integración	• Diseño de la arquitectura de repositorio en GitHub y desacoplamiento con Google Drive. • Implementación del pipeline modular en Python y wrappers de ejecución por cámara. • Desarrollo del módulo de color HSV y spread espacial en utils_color.py. • Automatización de la normalización temporal y filtros hstack en FFmpeg. • Orquestación del entorno de ejecución reproducible 'Zero-Touch' con gdown.
Marcos Andrés Wamba Ferreyra (Legajo 190802) Auditoría, Validación y Documentación	• Control de calidad cuadro por cuadro y validación cinemática de offsets en Avidemux/FFmpeg. • Auditoría de bounding boxes y delimitación de polígonos de exclusión en camara_rois.py. • Dimensionamiento de la arquitectura de hardware municipal (GPU/CPU/RAM para 15 cámaras). • Justificación analítica de los 15 FPS operativos y ahorro del 50% de carga de cómputo. • Análisis estadístico de los datasets generados y redacción integral del informe técnico.

10. Conclusiones y Hoja de Ruta para la Parte 2

La ejecución de la Parte 1 cumplió satisfactoriamente todos los objetivos técnicos establecidos:

- **Sincronización robusta:** Se normalizaron los tres streams urbanos eliminando derivas de HLS mediante recortes deterministas, verificados visualmente en compuestos sincronizados.
- **Viabilidad computacional comprobada:** Se demostró que 15 FPS es la frecuencia cinemática ideal, reduciendo el costo de inferencia al 50% y permitiendo operar 15 cámaras concurrentes en una única estación municipal.
- **Infraestructura moderna:** Se identificó a MediaMTX como la solución de transporte en tiempo real (<500 ms) para ingestión de IA con ahorro crítico de ancho de banda.
- **Filtrado espacial y cromático fidedigno:** camara_rois.py suprimió falsos positivos estáticos (contenedor de obra y monumento), mientras que el criterio HSV+Spread aisló exclusivamente los vehículos rojos reales.
- **Trazabilidad MLOps consolidada:** El entorno 'Zero-Touch' garantiza reproducibilidad independiente para los evaluadores, estructurando los resultados en eventos_total.csv.

Hoja de Ruta y Avance para la Parte 2:

La infraestructura desarrollada sienta las bases para las metas de la etapa siguiente:
1. Entrenamiento de detector especializado: Fine-tuning de YOLOv8 sobre dataset_v2 con 7 clases urbanas específicas (automóvil, utilitario, camión, pickup, persona, moto, bicicleta) utilizando los pesos preentrenados best.pt descargados en el pipeline.
2. Seguimiento multi-objeto (Tracking): Integración de algoritmos de seguimiento temporal (ByteTrack / BoT-SORT) para mantener track_ids únicos por vehículo a lo largo del tiempo.
3. Re-identificación inter-cámara: Vinculación de trayectorias globales a través de matrices de homografía y referencias urbanas espaciales compartidas.
4. Detección aérea móvil: Incorporación y evaluación de robustez del detector frente a secuencias de dron cenital (drone_ok.mp4).